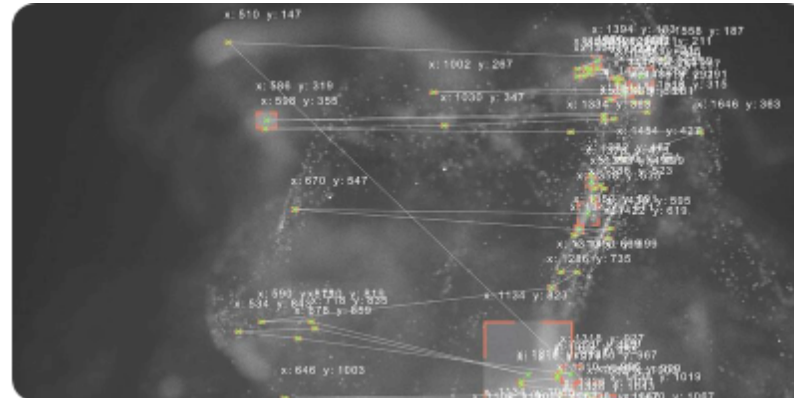
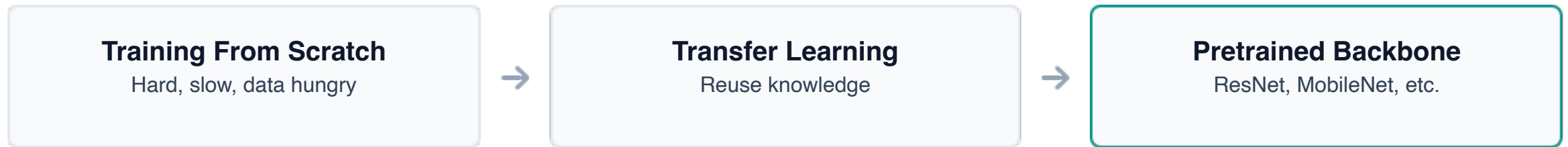


Feature Extraction vs. Fine-Tuning

How much of a pretrained model should we change?



Quick Recap: Where We Are

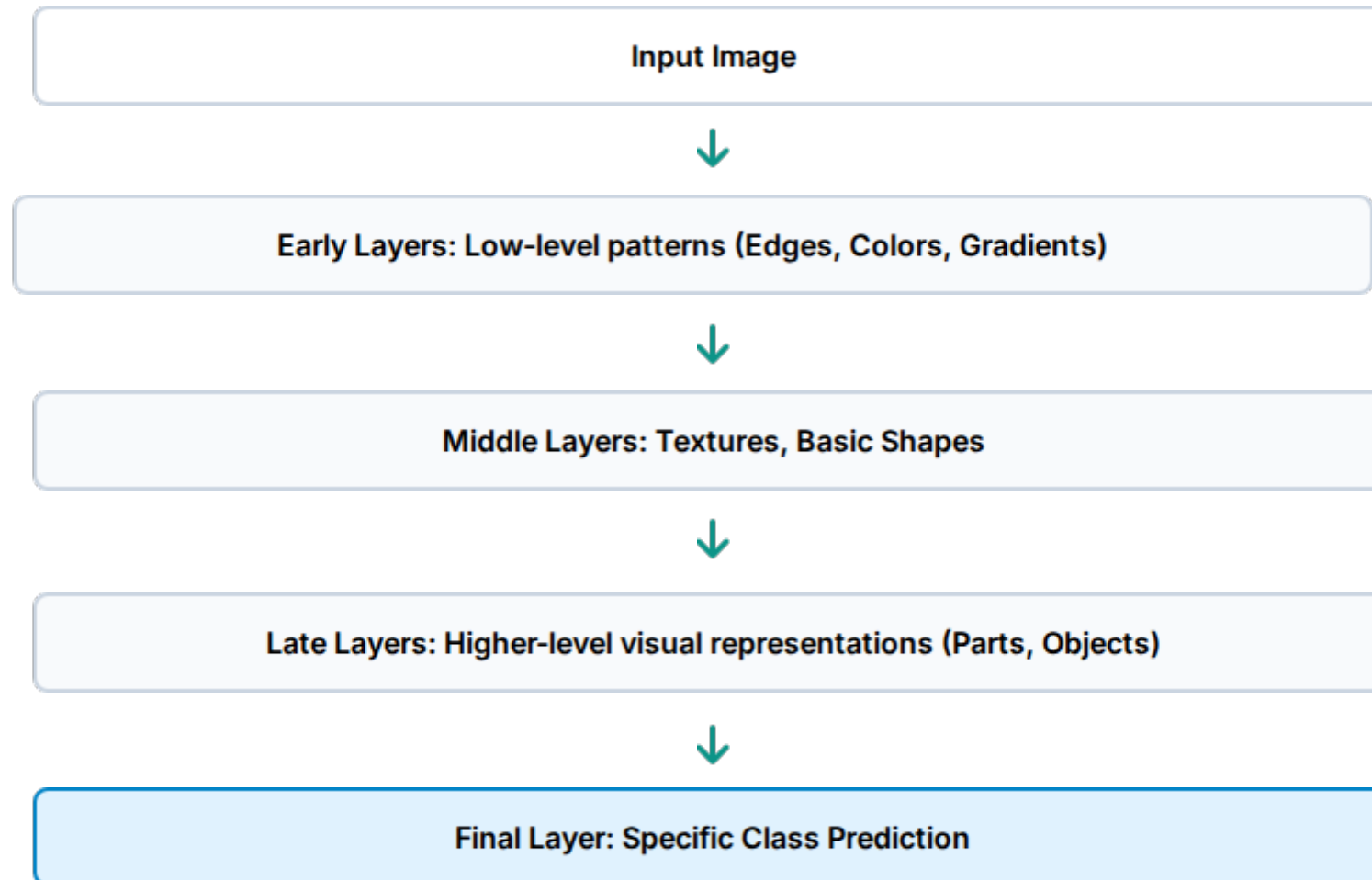


Now what do we actually train?

Learning Objectives

- 🎯 Define and differentiate **Feature Extraction** and **Fine-Tuning**.
- 🎯 Understand the concepts of **Freezing** and **Unfreezing** model layers.
- 🎯 Explain why the final classification layer is usually replaced and trained.
- 🎯 Choose appropriate freezing/unfreezing strategies based on dataset size and domain similarity.
- 🎯 Connect these concepts directly to the upcoming **Practical 5** assignment.

What Did the Pretrained CNN Learn?



Do we really need to retrain all those layers from scratch?

Maybe we can keep most of it completely unchanged.



What Is Feature Extraction?

"Feature extraction uses a pretrained model to generate useful features for a new task while keeping the pretrained feature extractor fixed."

Simple translation:

Reuse the visual knowledge. Train **ONLY** what is needed for the new task.

Feature Extraction: Visualized



Explain: Walk through this diagram slowly. This is the core architecture for Practical 5. The input goes through the frozen backbone, becomes a flat list of features, and is fed into a brand new, untuned classification head.

Analogy: The backbone is a camera lens (fixed). The head is the human looking at the photo deciding what it is (learning).

What Does "Frozen" Mean?

A frozen layer's parameters (weights and biases) are **not updated** during the training process. Gradient descent ignores them.

Frozen Layer

Weights *before* training

=

Weights *after* training

(The knowledge is perfectly preserved)

Trainable Layer

Weights *before* training

≠

Weights *after* training

(The network adapts and learns)

Explain: Define "Frozen" academically (gradients aren't applied) but rely heavily on the intuitive visual. Frozen means the numbers physically do not change in the computer's memory during `model.fit()`. **Time 2 mins**

Why Freeze the Backbone?



Preserves Knowledge

The model already learned highly useful visual features from millions of images.



Fewer Parameters

Training only the final layer reduces trainable parameters from millions to thousands.



Faster Training

Less math means each training epoch completes significantly faster, even on weaker hardware.



Prevents Overfitting

Highly useful when your new target dataset is very small and prone to memorization.

Why Change the Final Layer?

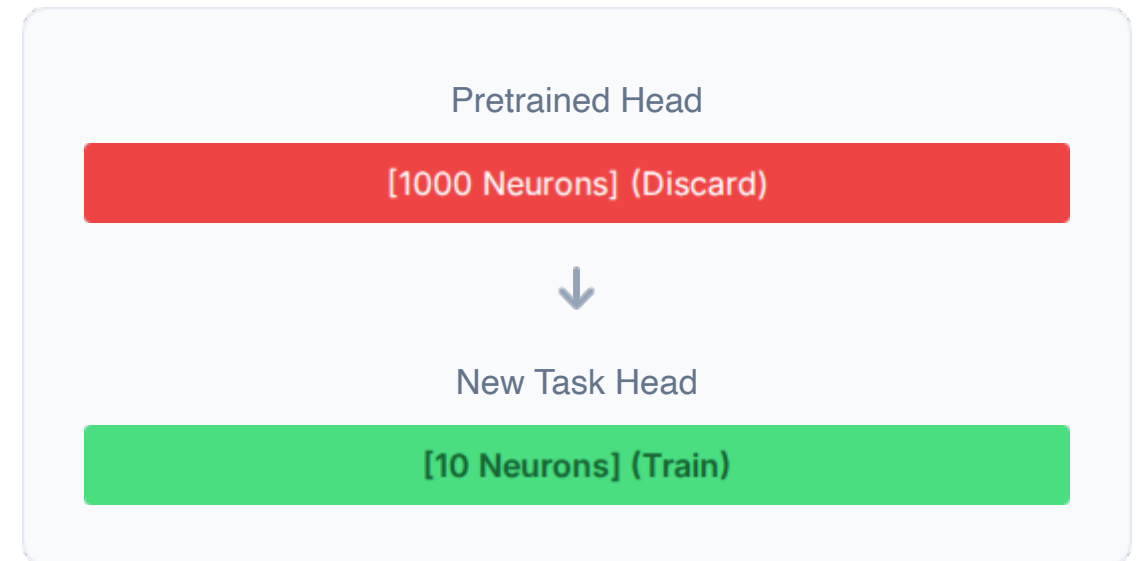
The Architecture Problem:

A pretrained model (like ResNet-50 on ImageNet) was originally built to output **1,000 distinct classes**.

But your new task might only have **10 classes** (e.g., classifying 10 types of vehicles).

Solution:

We literally chop off the final output layer, replace it with a new layer of size 10, and train just that new layer.



Explain: This is a structural necessity, not just an optimization. A matrix designed to output 1000 probabilities physically cannot output 10 probabilities without crashing. We must replace the head. **Time: 2 mins**

What If the New Problem Is Very Different?

Suppose our pretrained model learned by looking at **Natural Photographs** (cats, cars, trees).

Now, our new task is to analyze **Medical X-rays**.

Question:

Are the features extracted from natural photos guaranteed to be perfect for analyzing bone fractures?

Answer: Not necessarily.



Explain: Transition slide to Fine-Tuning. If the domain shifts drastically (Natural -> Medical/Satellite/Thermal), the frozen backbone's features might be slightly "off".

Class Question: "What happens if the feature extractor is looking for fur textures, but the input is an X-ray? It might extract useless noise."

Time: 2 mins

What Is Fine-Tuning?

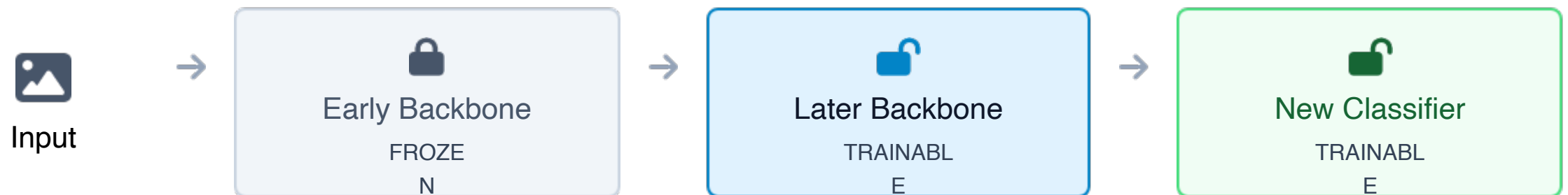
"Fine-tuning is continuing to train a pretrained model on a new dataset so that some of its learned parameters adapt to the new task."

Simple translation:

Reuse the old knowledge, but slightly **adapt (unfreeze)** parts of the backbone to fit the new problem.

Fine-Tuning: Visualized

Fine-tuning does **NOT** necessarily mean updating every single layer. We usually unfreeze only the later, task-specific layers.



Explain: Show how this differs from Feature Extraction. The blue block represents layers inside the backbone that we have unlocked. We keep early layers frozen (because edge detection is universal), but unfreeze later layers to adapt complex shapes.

Time: 3 mins

Direct Comparison

Concept	Feature Extraction	Fine-Tuning
Pretrained Backbone	 100% Frozen	 Some/All layers Trainable
New Classifier Head	 Trainable	 Trainable
Main Idea	Strictly reuse existing features	Adapt existing features to target
Training Computation Cost	Very Low	Moderate to High
Level of Target Adaptation	Lower	Higher

Explain: Walk through this table. This is the summary slide for an exam/quiz. Ensure students understand that BOTH methods train the new classifier head. The difference lies entirely in what happens to the backbone.

Time: 2 mins

Part 3: Strategies

What exactly does "Unfreezing" mean?

Unfreezing means instructing the deep learning framework (like PyTorch or TensorFlow) to allow previously frozen parameters to be updated by backpropagation during training.

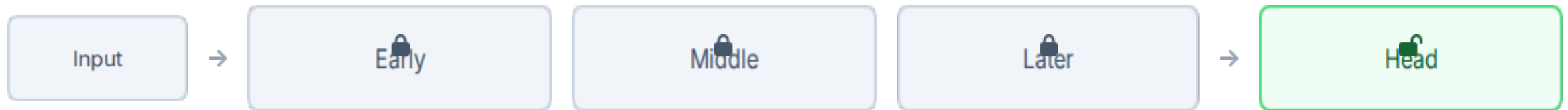


"You don't rebuild your entire skill set. You just unlock the parts that need improvement."

Explain: Introduce the concept of Freezing/Unfreezing strategies. In code, this is literally a boolean flag on the layers. We are deciding *which* layers get this flag set to True.

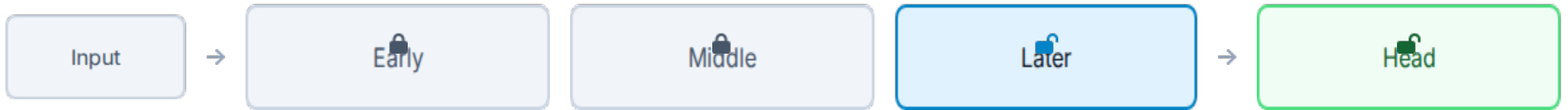
Time: 2 mins

Strategy 1: Freeze the Entire Backbone



- ✓ **What it is:** Pure feature extraction.
- ✓ **How it works:** Train ONLY the new classification head.
- ✓ **When to use:** Best as a simple starting baseline, particularly when your target dataset is very small and prone to overfitting.

Strategy 2: Unfreeze the Last Few Layers

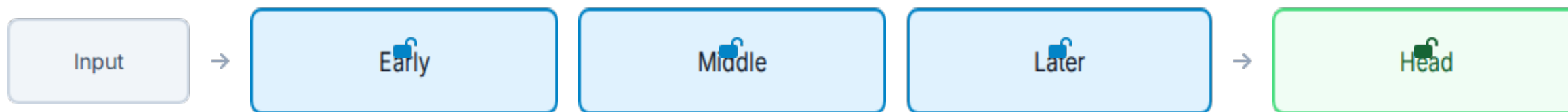


- ✓ **What it is:** Partial fine-tuning.
- ✓ **How it works:** Keep general features (edges/shapes) fixed, but allow the most highly task-specific features (parts/objects) to adapt.
- ✓ **When to use:** The target task is somewhat different from the source, and you have enough data to train a few layers without overfitting.

Explain: This is the most common industry approach for fine-tuning. Unfreeze the top block of ResNet or EfficientNet. It balances preserving basic math with adapting semantic math.

Time: 2 mins

Strategy 3: Unfreeze More (or All) Layers

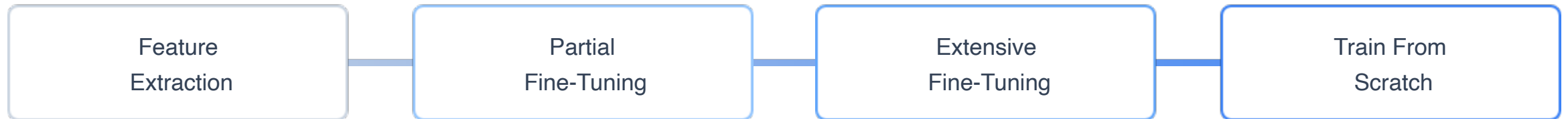


- ⚠️ **What it is:** Extensive fine-tuning.
- ⚠️ **Pros:** Maximum adaptation. Can be highly useful when the target problem differs drastically from the source (e.g., Medical, Satellite).
- ⚠️ **Cons:** High computation. Severe risk of overfitting unless you have a massive dataset.

Explain: Unfreezing everything means the entire network is shifting. Use words like "can" and "may". If you do this with 1000 images, the network will instantly overfit and destroy its pretrained knowledge. Time 2 mins

The Transfer Learning Spectrum

Choosing a strategy is not a binary choice. It is a spectrum of how much pretrained knowledge we choose to modify.



More Layers Frozen More Parameters Trainable

Part 4: How Do We Choose? (Dataset Size)

Note: These are engineering guidelines, not strict mathematical rules.

↗ Small Dataset

If you have very little data, fine-tuning many layers will almost certainly lead to overfitting.



Usually start with: More frozen layers (Feature Extraction).

🗄 Larger Dataset

If you have a large dataset, the network has enough examples to safely update internal representations.



Can consider: Unfreezing more layers (Fine-Tuning).

How Do We Choose? (Domain Similarity)

More Similar

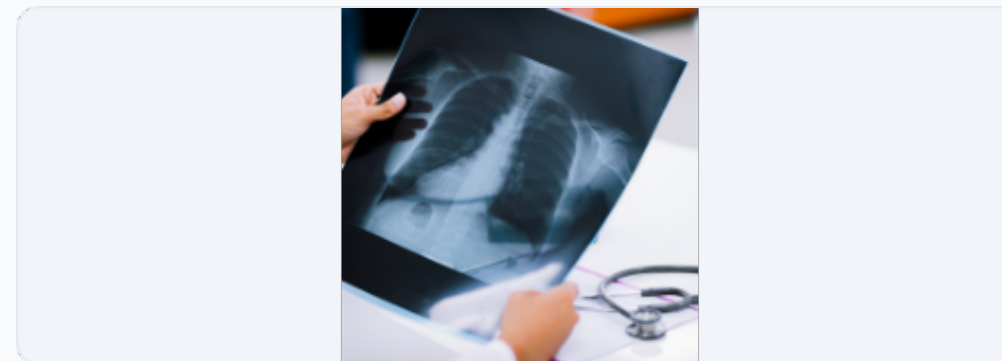
Source: Natural Photos → Target: Natural Photos



The pretrained features are already excellent for this task. **Feature extraction** or very light fine-tuning is usually sufficient.

More Different

Source: Natural Photos → Target: Medical X-rays

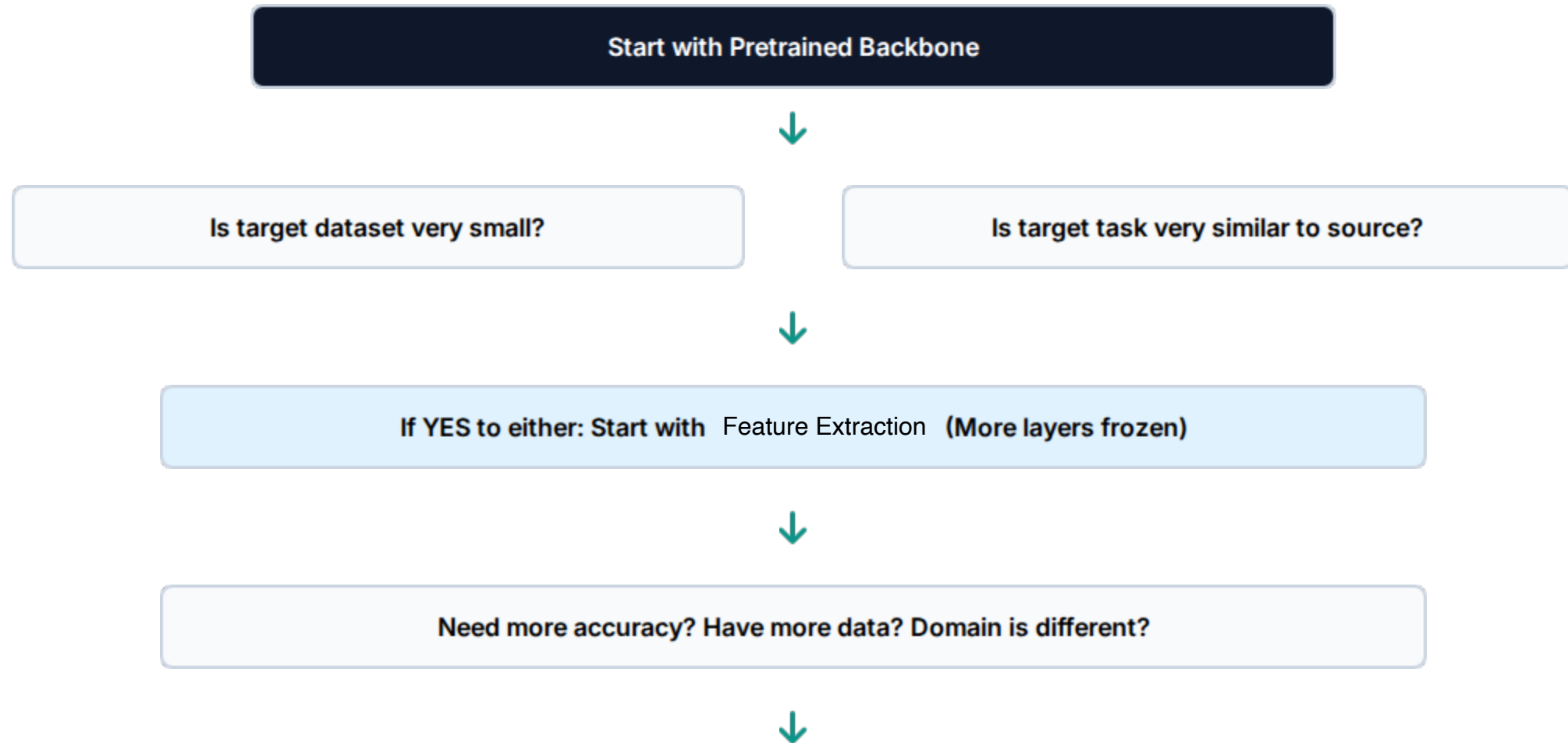


The pretrained features may be less applicable. Adapting deeper features via **Fine-Tuning** may become more useful.

Explain: The second factor is similarity. If the new images look totally alien to the pretrained network (like ultrasound or satellite), the backbone needs more adaptation to be useful.

Time: 2 mins

Simple Decision Framework



Explain: Walk through this flowchart. Emphasize the Golden Rule: We don't guess. We start simple (Feature Extraction), test the accuracy, and if we need more (and have the data), we unfreeze **1 layer and test again**.

Partial Fine-Tuning

Part 5: Why Fine-Tuning Needs Care

A pretrained model already contains highly useful, delicate mathematical knowledge.

If we use a massive **Learning Rate** during fine-tuning, the large updates can violently overwrite and damage these useful representations.

"If you already know how to drive, learning a new route doesn't require relearning how to operate the steering wheel."

Small Updates (Low LR)

Safer adaptation. Gently nudges existing weights toward the new task.

Large Updates (High LR)

Greater risk. Can cause "Catastrophic Forgetting" of the useful pretrained features.

Explain: Preview of next lecture's learning rate discussion. Keep it intuitive. You don't use a sledgehammer to tune a piano. You use a very small, precise tool (small learning rate).

Time: 3 mins

Analogy: The driving analogy is key here. Fine-tuning is learning the new route, not re-learning the pedals.

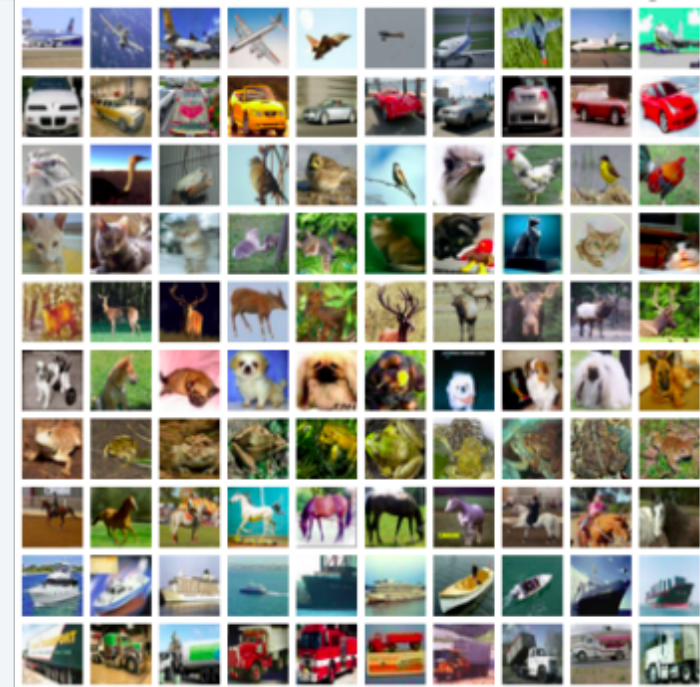
Part 6: Preparing for Practical 5

Your Assignment: Feature Extraction

Goal: Use a pretrained backbone as a feature extractor, and train a new classifier head on the CIFAR-10 dataset (or custom dataset).

Workflow:

- 1 Load Dataset (e.g., CIFAR-10).
- 2 Load Pretrained Backbone (e.g., ResNet18).
- 3 Freeze the backbone parameters.
- 4 Replace the Final Classifier Head.
- 5 Train ONLY the new head.



Practical 5: The Experiment

During inference (prediction), the input flows through the **entire network**. But during training, we only update the **trainable parameters**.

The Challenge

Run two versions of your code and compare the results:

- **Version A:** 100% Frozen Backbone (Feature Extraction).
- **Version B:** Unfreeze the final few layers (Partial Fine-Tuning).

What to Measure

Observe the trade-offs. Record and compare:

- **Accuracy:** Did fine-tuning help or hurt?
- **Training Time:** How much longer did Version B take per epoch?
- **Trainable Parameters:** Check the model summary output.

Explain: Give them the mindset of an engineer. We don't just write code; we run experiments to measure trade-offs. Ask them to hypothesize which version will be faster, and which might be more accurate. **Time: 3 mins**

Final Summary

Feature Extraction:

 Backbone frozen.  Train classifier.

Freezing:

Parameters do not change.

Strategy:

Start simple → evaluate → gradually adapt.

Fine-Tuning:

 Some pretrained layers updated.

Unfreezing:

Parameters can change.

Exit Question:

If you have a very small dataset and a pretrained CNN, would you immediately unfreeze the entire network? Why or why not?

Next lecture: Layer-wise learning rates and domain adaptation

Explain: Review the core terminology rapidly. Then pose the exit question.

Time: 2 mins

Class Question: Call on a student to answer the exit question. (Answer: No, it will likely overfit and destroy the pretrained weights because a small dataset lacks the statistical variation needed to safely update millions of parameters).

Image Sources

600 × 400

https://hackster.imgix.net/uploads/attachments/1118120/screen_shot_2020-04-30_at_2_03_44_pm_TfM4bFvUik.png

Source: hackster.io

600 × 400

<https://kims.org.uk/wp-content/uploads/2024/11/x-ray-scan.png>

Source: kims.org.uk

600 × 400

https://upload.wikimedia.org/wikipedia/en/f/fc/Cats_%26_Dogs_film.jpg?utm_source=en.wikipedia.org&utm_campaign=index&utm_content=original

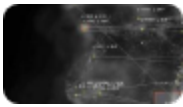
Source: en.wikipedia.org

600 × 400

https://miro.medium.com/max/1434/1*Syi6h2P0A46Z5hV6LgXzBw.png

Source: colab.research.google.com

Image Sources



<https://static.vecteezy.com/system/resources/thumbnails/072/165/629/large/computer-vision-with-object-recognition-scanner-overlay-big-data-cctv-ai-interface-scanning-blob-tracking-lines-coordinates-bounding-boxes-plexus-style-connections-abstract-flow-black-n-white-video.jpg>

Source: www.vecteezy.com



<https://kims.org.uk/wp-content/uploads/2024/11/x-ray-scan.png>

Source: kims.org.uk



https://upload.wikimedia.org/wikipedia/en/f/fc/Cats_%26_Dogs_film.jpg?utm_source=en.wikipedia.org&utm_campaign=index&utm_content=original

Source: en.wikipedia.org



<https://storage.googleapis.com/kaggle-media/competitions/kaggle/3649/media/cifar-10.png>

Source: www.kaggle.com